

GUÍA PASO A PASO — PROBADO Y VERIFICADO

OpenClaw + Ollama en Hostinger VPS

Cómo conectar un modelo de IA local (gratis) con tu agente de OpenClaw en un VPS de Hostinger con la instalación 1-Click de Docker.

Fecha: 5 abril 2026

Tiempo estimado: 15-20 minutos

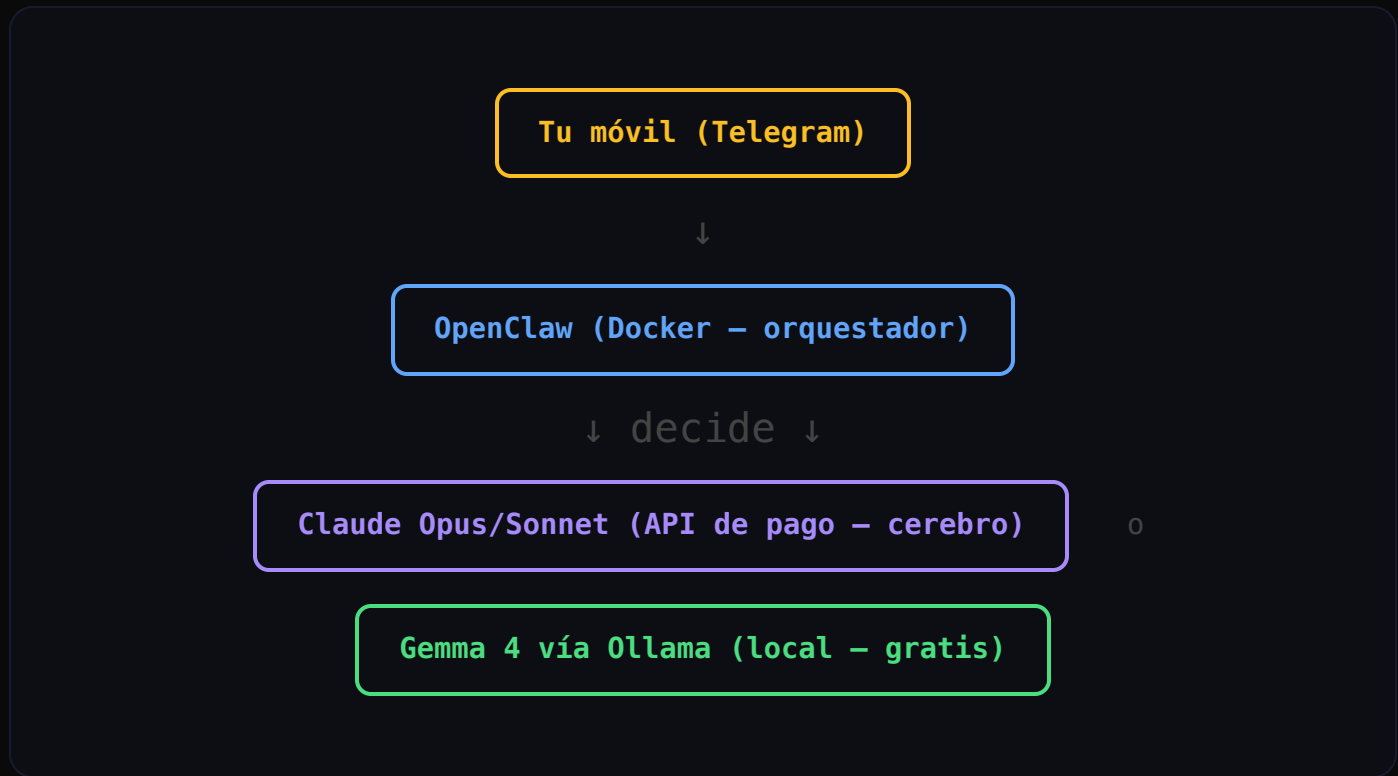
Requisito: VPS KVM 4+ (16 GB RAM)

ÍNDICE

- | | |
|------------------------------------|--|
| 01. Qué vamos a montar | 07. Paso 4 — Verificar la conexión |
| 02. Requisitos previos | 08. Paso 5 — Configurar OpenClaw |
| 03. Qué modelo elegir según tu VPS | 09. Paso 6 — Probar desde Telegram |
| 04. Paso 1 — Instalar Ollama | 10. Seguridad — Qué NO mostrar en pantalla |
| 05. Paso 2 — Descargar el modelo | 11. Errores comunes y soluciones |
| 06. Paso 3 — Abrir Ollama a Docker | 12. Resumen final |

Qué vamos a montar

Vamos a crear un sistema donde tu agente de IA tiene **dos cerebros**: uno de pago (Claude) para tareas complejas, y uno gratuito (Gemma 4 vía Ollama) para tareas simples. Ambos corriendo 24/7 en tu servidor.



¿Por qué dos modelos? Claude es más inteligente pero cuesta dinero (pagas por token). Gemma 4 es gratis pero más lento y menos capaz. OpenClaw usa Claude como modelo principal y Gemma 4 como fallback — si Claude falla o quieres ahorrar, Gemma 4 responde sin coste.

El truco técnico

OpenClaw corre **dentro de Docker** (así lo instala Hostinger con 1-Click). Ollama corre **fuera de Docker**, en el servidor directamente. Por defecto, no se ven entre sí. El 90% del trabajo de esta guía es **conectarlos**.

Requisitos previos

- **VPS de Hostinger** con OpenClaw instalado via 1-Click (Docker)

- **Mínimo 16 GB de RAM** (plan KVM 4 o superior)
- **Acceso SSH** al VPS (terminal de hPanel o desde tu ordenador)
- **OpenClaw funcionando** con Claude configurado y Telegram conectado

¿Por qué 16 GB mínimo? Gemma 4 E4B necesita ~5 GB de RAM para ejecutarse. Con 8 GB de VPS, el sistema operativo + Docker + OpenClaw consumen ~5.5 GB, dejando solo 2.5 GB libres — insuficiente. Con 16 GB tienes 14 GB libres, sobra de sobra.

Qué modelo elegir según tu VPS

MODELO	DESCARGA	RAM NECESARIA	VELOCIDAD (CPU)	PLAN MÍNIMO
gemma4:e4b ★	~5 GB	~5 GB	~40 segundos	KVM 4 (16 GB)
gemma4:e2b	~7 GB	~7.3 GB	~1 min 40 seg	KVM 4 (16 GB)
qwen3:1.7b	~1.5 GB	~2 GB	~10 segundos	KVM 2 (8 GB)

Recomendación: `gemma4:e4b` es el mejor equilibrio entre calidad y velocidad para un VPS de 16 GB sin GPU. Es Gemma 4 de Google, gratuito, y responde en ~40 segundos.

Sobre la velocidad: Un VPS de Hostinger no tiene GPU — todo se procesa en CPU. Por eso los modelos locales tardan más que en tu ordenador personal (que sí tiene GPU). Esto es normal y esperado.

Paso 1 — Instalar Ollama

Conéctate a tu VPS por SSH (desde la terminal de hPanel o desde tu ordenador) y ejecuta:

1

Instalar Ollama con un solo comando

```
curl -fsSL https://ollama.com/install.sh | sh
```

Esto descarga e instala Ollama automáticamente. Tarda menos de 1 minuto.

2

Verificar que está corriendo

```
systemctl status ollama
```

Debes ver `active (running)` en verde. Si no, ejecuta `systemctl start ollama`.

¿Qué es Ollama? Es como el "Docker de los modelos de IA". Te permite descargar y ejecutar modelos de IA en tu servidor, sin conexión a internet, sin pagar APIs, sin enviar tus datos a ningún sitio. Piensa en Netflix (Claude, pagas streaming) vs descargar la película (Ollama, gratis).

Paso 2 — Descargar el modelo

3

Descargar Gemma 4 E4B

```
ollama pull gemma4:e4b
```

Descarga ~5 GB. Tarda 2-5 minutos dependiendo de la conexión del VPS.

4

Verificar que se descargó

```
curl http://127.0.0.1:11434/api/tags
```

Debes ver `gemma4:e4b` en la respuesta JSON. Si aparece, el modelo está listo.

Paso 3 — Abrir Ollama a Docker

Este es el paso más importante. Por defecto, Ollama solo acepta conexiones de sí mismo (`127.0.0.1`). Pero OpenClaw corre dentro de Docker, en otra red. Necesitamos decirle a Ollama: "acepta conexiones de Docker también".

5

Configurar Ollama para escuchar en todas las interfaces

```
# Crear el archivo de configuración
mkdir -p /etc/systemd/system/ollama.service.d

echo '[Service]
Environment="OLLAMA_HOST=0.0.0.0"' > /etc/systemd/system/ollama.service.d/override.conf

# Recargar y reiniciar
```

```
sudo systemctl daemon-reload
sudo systemctl restart ollama
```

6

Verificar que escucha en todas las interfaces

```
ss -tlnp | grep 11434
```

Debes ver `*:11434` (todas las interfaces). Si dice `127.0.0.1:11434`, el cambio no se aplicó — repite el paso 5.

Correcto: `LISTEN 0 4096 *:11434 *:*`

Incorrecto: `LISTEN 0 4096 127.0.0.1:11434 0.0.0.0:*`

Paso 4 — Verificar la conexión Docker → Ollama

Ahora necesitamos saber la dirección IP que usa Docker para comunicarse con el servidor. Es la "puerta de enlace" (gateway) de la red Docker.

7

Averiguar la gateway de Docker

```
# Primero, averigua el nombre de la red
docker network ls

# Busca la red de OpenClaw (tipo: openclaw-XXXX_default)
```

```
# Luego inspecciona para ver la gateway  
docker network inspect openclaw-XXXX_default | grep Gateway
```

Te dará algo como `"Gateway": "172.18.0.1"`. **Apunta esta IP** — la necesitas en el siguiente paso.

8

Probar que OpenClaw puede ver a Ollama

```
# Reemplaza NOMBRE_CONTENEDOR y GATEWAY_IP con tus valores  
docker exec -it NOMBRE_CONTENEDOR curl http://GATEWAY_IP:11434/api/tags
```

Si ves `gemma4:e4b` en la respuesta, **la conexión funciona**. Si dice "Could not connect", revisa el paso 3.

Ejemplo real:

```
docker exec -it openclaw-hkqb-openclaw-1 curl http://172.18.0.1:11434/api/tags  
{"models": [{"name": "gemma4:e4b", ...}]}
```

¿Por qué no usar **127.0.0.1**? Porque desde dentro de Docker, `127.0.0.1` apunta al propio contenedor, no al servidor. La gateway (`172.18.0.1`) es la dirección del servidor visto desde Docker.

Paso 5 — Configurar OpenClaw

Aquí es donde muchos tutoriales fallan. La imagen de Hostinger tiene un comportamiento especial: **sobreescribe la configuración durante los primeros ~15 segundos tras arrancar**. Por eso hay que esperar antes de escribir la config.

Regla de oro: Siempre reinicia el contenedor PRIMERO, espera 20 segundos, y DESPUÉS escribe la configuración. Si lo haces al revés, el gateway sobreescribe tus cambios.

9

Añadir Ollama a los plugins permitidos + configurar el provider

Ejecuta este comando **exacto** (reemplaza `NOMBRE_CONTENEDOR` y `GATEWAY_IP`):

```
docker restart NOMBRE_CONTENEDOR && sleep 20 && docker exec -it NOMBRE_CO
const fs = require('fs');
const cfg = JSON.parse(fs.readFileSync('/data/.openclaw/openclaw.json', 'u

// 1. Añadir ollama a plugins permitidos
if (!cfg.plugins.allow.includes('ollama')) {
  cfg.plugins.allow.push('ollama');
}

// 2. Configurar el provider de Ollama
cfg.models = cfg.models || {};
cfg.models.mode = 'merge';
cfg.models.providers = cfg.models.providers || {};
cfg.models.providers.ollama = {
  baseUrl: 'http://GATEWAY_IP:11434/v1',
  apiKey: 'ollama-local',
  api: 'openai-completions',
  models: [{
    id: 'gemma4:e4b',
    name: 'Gemma 4 E4B',
    reasoning: false,
    contextWindow: 131072,
    maxTokens: 8192,
```



```
    cost: { input: 0, output: 0, cacheRead: 0, cacheWrite: 0 }
  }]
};

// 3. Añadir como fallback del agente
if (cfg.agents && cfg.agents.list && cfg.agents.list[0]) {
  cfg.agents.list[0].model = cfg.agents.list[0].model || {};
  cfg.agents.list[0].model.fallbacks = ['ollama/gemma4:e4b'];
}

fs.writeFileSync('/data/.openclaw/openclaw.json', JSON.stringify(cfg, null, 2));
console.log('OK: Ollama configurado correctamente');
"
```

Campos obligatorios del provider (si te falta uno, falla):

- `baseUrl` — URL de Ollama **con** `/v1` al final (modo OpenAI-compatible)
- `apiKey` — Cualquier valor, `"ollama-local"` es la convención
- `api` — Debe ser `"openai-completions"`
- `models` — Array con al menos un modelo, cada uno con `id`, `name`, `cost` (con los 4 subcampos)

10

Verificar que la configuración se aplicó

```
docker logs NOMBRE_CONTENEDOR --tail 5 2>&1 | grep -i "reload"

# Debes ver algo como:
[reload] config hot reload applied (models.providers.ollama.models...)
```

Si ves "config hot reload applied" — la configuración se aceptó. Si ves "invalid config" — revisa el formato del JSON.

11

Confirmar la configuración completa

```
docker exec -it NOMBRE_CONTENEDOR node -e "  
const cfg = JSON.parse(require('fs').readFileSync('/data/.openclaw/openc  
console.log('=== MODELOS CONFIGURADOS ===');  
console.log('Primary:', cfg.agents.list[0].model.primary);  
console.log('Fallbacks:', cfg.agents.list[0].model.fallbacks);  
console.log('');  
console.log('=== OLLAMA PROVIDER ===');  
const o = cfg.models.providers.ollama;  
console.log('URL:', o.baseUrl);  
console.log('API:', o.api);  
console.log('Modelo:', o.models[0].id, '(' + o.models[0].name + ')');  
console.log('Contexto:', o.models[0].contextWindow, 'tokens');  
console.log('Coste:', JSON.stringify(o.models[0].cost));  
"
```

Salida esperada:

```
=== MODELOS CONFIGURADOS ===  
Primary: Claude Sonnet 4.6  
Fallbacks: [ 'ollama/gemma4:e4b' ]  
  
=== OLLAMA PROVIDER ===  
URL: http://172.18.0.1:11434/v1  
API: openai-completions  
Modelo: gemma4:e4b (Gemma 4 E4B)
```

Contexto: 131072 tokens

Coste: {"input":0,"output":0,"cacheRead":0,"cacheWrite":0}

Paso 6 — Probar desde Telegram

12

Seleccionar el modelo de Ollama

En Telegram, escribe al bot:

```
/model ollama/gemma4:e4b
```

Debes ver:  Model changed to ollama/gemma4:e4b

13

Enviar un mensaje

Escribe cualquier mensaje al bot. Aparecerá "typing..." durante ~40 segundos (es normal en CPU) y luego responderá.

La primera vez tarda más. La primera petición a Gemma 4 puede tardar 1-2 minutos porque Ollama tiene que cargar el modelo completo en RAM (~5 GB). Las siguientes respuestas serán más rápidas (~40 segundos).

14

Volver a Claude

Para cambiar de vuelta a Claude:

```
/model anthropic/claude-sonnet-4-6
```

¡Listo! Ahora tienes dos modelos de IA corriendo 24/7 en tu servidor. Claude como cerebro principal (calidad máxima) y Gemma 4 como alternativa gratuita (coste \$0).

Seguridad — Qué NO mostrar en pantalla

Si estás grabando esto para un vídeo o compartiéndolo en pantalla, **NUNCA** muestres estos datos:

- ❌ **La IP pública del VPS** — con ella pueden escanear puertos e intentar atacarte
- ❌ **La IPv6 del VPS** — mismo riesgo
- ❌ **El archivo `.env`** — contiene TODAS tus API keys (Anthropic, OpenAI, Brave...)
- ❌ **Los tokens de Telegram/Slack** en `openclaw.json` — pueden controlar tu bot
- ❌ **El `OPENCLAW_GATEWAY_TOKEN`** — da acceso al dashboard
- ✅ **Los comandos de instalación** — son genéricos, no hay riesgo
- ✅ **La salida de verificación** — no contiene datos sensibles
- ✅ **La conversación con el bot** — es contenido público

Consejo para el vídeo: Personaliza el prompt de la terminal antes de grabar para que no muestre el hostname real:

```
export PS1="juanpe@mi-servidor:~$ "
```

Errores comunes y soluciones

ERROR	CAUSA	SOLUCIÓN
<code>Could not connect to 172.18.0.1 port 11434</code>	Ollama solo escucha en 127.0.0.1	Repetir Paso 3 — configurar <code>OLLAMA_HOST=0.0.0.0</code>
<code>Invalid config: models.providers.ollama.models</code>	Falta el array de <code>models</code> o campos obligatorios	Revisar que el JSON tenga <code>id</code> , <code>name</code> , <code>cost</code> con los 4 subcampos
<code>HTTP 404: page not found</code>	Se usa <code>api: "ollama"</code> con URL sin <code>/v1</code>	Usar <code>api: "openai-completions"</code> con <code>/v1</code> en la URL
<code>500 model requires more system memory</code>	El modelo no cabe en la RAM disponible	Usar un modelo más pequeño o ampliar el VPS
<code>400 model does not support tools</code>	El modelo no soporta tool calling	Usar <code>gemma4:e4b</code> o <code>qwen3:1.7b</code> que sí soportan tools
<code>typing TTL reached (2m)</code>	El modelo tardó más de 2 minutos en responder	Usar un modelo más pequeño y rápido, o un VPS con más CPU
<code>Model not allowed</code> en Telegram	Falta <code>"ollama"</code> en <code>plugins.allow</code>	Añadir <code>"ollama"</code> al array de plugins permitidos (Paso 5)

ERROR	CAUSA	SOLUCIÓN
La config se borra al reiniciar	El gateway sobreescribe el JSON al arrancar	Siempre esperar 20 segundos después del reinicio antes de escribir la config

Resumen final

6

PASOS TOTALES

\$0

COSTE DEL MODELO
LOCAL

~40s

TIEMPO DE RESPUESTA

24/7

DISPONIBILIDAD

Lo que tienes ahora

- **OpenClaw** corriendo 24/7 en tu VPS de Hostinger
- **Claude** como modelo principal (inteligente, de pago)
- **Gemma 4** como fallback gratuito (0 coste por token)
- **Telegram** como mando a distancia para hablar con tu agente
- **Cambio de modelo en 1 segundo** con `/model` en Telegram

Comandos de referencia rápida

```
# Ver modelos disponibles (desde Telegram)
/model
```

```
# Cambiar a Gemma 4 (gratis)
/model ollama/gemma4:e4b

# Cambiar a Claude (de pago)
/model anthropic/claude-sonnet-4-6

# Ver estado de Ollama (desde SSH)
systemctl status ollama

# Ver modelos descargados en Ollama
ollama list

# Ver RAM usada
free -h

# Ver si Ollama está procesando
top -bn1 | grep ollama
```

Para el vídeo: "Mi ordenador está apagado y mi agente de IA sigue trabajando. Con dos cerebros: uno inteligente de pago, otro gratuito. 24 horas al día, 7 días a la semana."

Guía creada por Juanpe Navarro (@juanpe.divisual) — Abril 2026

Probado y verificado en Hostinger VPS KVM 4 (16 GB RAM) con OpenClaw 2026.3.13